LẬP TRÌNH WEB NÂNG CAO · BÀI THỰC HÀNH

Trình phát nhạc mini

Bài thực hành TH02 – Lập trình Web nâng cao · Thời gian: **4 tiếng** · Trọng tâm: **Buổi 4** (Zustand, Context nâng cao, middleware) · **Buổi 5** (tối ưu hiệu năng) · **Buổi 6** (kiểm thử)

Lưu ý nộp bài

- Mỗi bạn tạo 1 repo trên GitHub, đặt tên **<Mã sinh viên>_LTWNC_TH02**, VD **B25DCCC010_LTWNC_TH02**.
- Repo để **public**, không để private. Làm bài cá nhân, không sao chép bài của nhau.
- Làm xong mỗi phần thì commit một lần với nội dung **Phan 1**, **Phan 2**, **Phan 3**, **Phan 4**.
- Trước khi nộp, cả 2 lệnh `npm run build` và `npm test` phải chạy thành công.

Bối cảnh

Buổi thực hành trước các em đã dùng Redux Toolkit để quản lý deadline. Buổi này các em xây một **"Trình phát nhạc mini"** giống Spotify thu nhỏ: thư viện **10.000 bài hát** cuộn mượt, thanh phát nhạc chạy theo từng giây, danh sách yêu thích được ghi nhớ và các playlist tự tạo. Không có âm thanh thật: "phát nhạc" chỉ là đồng hồ đếm giây và thanh tiến trình.

Với 10.000 dòng dữ liệu và một đồng hồ cập nhật mỗi giây, nếu viết không cẩn thận ứng dụng sẽ giật rõ rệt. Vì vậy bài này tập trung vào các nội dung của Buổi 4–6: chọn đúng công cụ quản lý state, tối ưu render và viết test. **Không dùng Redux Toolkit** trong bài này.

Địa chỉ (URL)	Trang	Làm ở
`/`	Thư viện bài hát	Phần 1
`/favorites`	Bài hát yêu thích	Phần 2
`/playlists`	Danh sách playlist	Phần 3
`/playlists/:id`	Chi tiết playlist	Phần 3
Mọi trang	Thanh phát nhạc cố định ở đáy màn hình	Phần 2

Chuẩn bị project

Yêu cầu Node.js 22.12 trở lên (bản 22 hoặc 24 LTS). Kiểm tra bằng `node -v`; bản Vite và Vitest hiện tại không chạy được trên Node 20 trở xuống.

```
npm create vite@latest music-player -- --template react-ts
cd music-player
npm install
npm install zustand react-router-dom@6 react-window@1
npm install -D @types/react-window@1 vitest jsdom @vitest/coverage-v8
npm install -D @testing-library/react @testing-library/dom
npm install -D @testing-library/jest-dom @testing-library/user-event
```

Thêm vào `package.json` 2 script: `"test": "vitest run"` và `"coverage": "vitest run --coverage"`. Vitest có API gần như giống hệt Jest đã học ở Buổi 6 (`describe` , `it` , `expect`), chỉ khác: `jest.fn()` →

`vi.fn()`, `jest.useFakeTimers()` → `vi.useFakeTimers()`, và phải import các hàm này từ `vitest`.

File `vite.config.ts` (thay toàn bộ nội dung):

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: './src/setupTests.ts',
    coverage: { include: ['src/features/**', 'src/utils/**'] },
  },
});
```

File `src/setupTests.ts`:

```
import '@testing-library/jest-dom/vitest';
import { cleanup } from '@testing-library/react';
import { afterEach } from 'vitest';

afterEach(() => cleanup());
```

Lưu ý với template React + TypeScript của Vite: import kiểu phải viết `import type { ... }`; không dùng `enum` (dùng union literal như `'Pop' | 'Rock'`); biến hoặc import không dùng tới sẽ làm `npm run build` báo lỗi. Không dùng `any`.

Dữ liệu cho sẵn

File `src/types.ts` (các em được thêm kiểu mới vào file này):

```
export type Genre = 'Pop' | 'Ballad' | 'Rock' | 'Indie' | 'Rap';

export interface Song {
  id: string;
  title: string;
  artist: string;
  genre: Genre;
  duration: number; // đơn vị: giây
}
```

File `src/data/songs.ts` (copy nguyên văn): hàm sinh dữ liệu giả lập, luôn cho ra cùng một bộ bài hát.

```
import type { Song, Genre } from '../types';

const ADJ = ['Mưa', 'Nắng', 'Gió', 'Đêm', 'Phố', 'Biển', 'Mây', 'Sao'];
const NOUN = [
  'tháng Sáu', 'cuối ngày', 'Hà Nội', 'mùa thu', 'xa xôi',
  'bình yên', 'ngày cũ', 'lặng lẽ', 'trong tim', 'mùa hạ',
];
const ARTISTS = [
  'Lê Minh', 'Hà Anh', 'Tuấn Vũ', 'Ngọc Mai', 'Bảo Trâm',
  'Đức Phúc', 'Thu Hà', 'Quang Huy', 'Mỹ Linh', 'Hoàng Long',
];
export const GENRES: Genre[] = ['Pop', 'Ballad', 'Rock', 'Indie', 'Rap'];

export const generateSongs = (count: number): Song[] =>
  Array.from({ length: count }, (_, i) => {
    const name = `${ADJ[i % 8]} ${NOUN[Math.floor(i / 8) % 10]}`;
    return {
      id: `s${i + 1}`,
      title: `${name} ${Math.floor(i / 80) + 1}`,
      artist: ARTISTS[(i * 7) % 10],
      genre: GENRES[(i * 3) % 5],
      duration: 150 + ((i * 37) % 151), // từ 150 đến 300 giây
    };
  });
```

Trong ứng dụng, sinh **một lần** 10.000 bài: `export const SONGS = generateSongs(10000);`. Vài bài đầu: **s1** "Mưa tháng Sáu 1" – Lê Minh – Pop – 150 giây; **s2** "Nắng tháng Sáu 1" – Quang Huy – Indie – 187 giây; **s3** "Gió tháng Sáu 1" – Bảo Trâm – Ballad – 224 giây.

Thời lượng hiển thị dạng **phút:giây** bằng hàm tự viết `formatDuration(seconds: number): string` trong `src/utils/format.ts`: 150 → **2:30**, 65 → **1:05**, 0 → **0:00**.

Phần 1 – Thư viện 10.000 bài hát (khoảng 60 phút)

Buổi 5: virtualization, React.memo, useMemo, useCallback, debounce, key ổn định.

1. **Khung ứng dụng**: React Router v6 với các route ở bảng trên; thanh menu gồm **Thư viện**, **Yêu thích**, **Playlist** (NavLink, mục đang xem được tô sáng).
2. **Danh sách ảo hóa**: trang Thư viện hiển thị danh sách bằng `FixedSizeList` của **react-window** (cao 560px, mỗi dòng 56px). Mỗi dòng (`SongRow`) gồm: số thứ tự, tên bài, ca sĩ, thể loại, thời

lượng, và 3 nút **Phát**, ♥ (Phần 2), **+ Hàng đợi** (Phần 2). Dùng `song.id` làm key, không dùng index.

3. **Tìm kiếm có debounce**: ô **Tìm kiếm** (có `<label>`) tìm theo **tên bài hoặc tên ca sĩ**, không phân biệt hoa thường. Viết hook generic `useDebounce<T>(value: T, delay: number): T` trong `src/hooks/useDebounce.ts`; danh sách chỉ lọc lại sau khi ngừng gõ **300ms**.
4. **Lọc và sắp xếp**: ô chọn **Thể loại** (Tất cả, Pop, Ballad, Rock, Indie, Rap) và ô chọn **Sắp xếp** (Mặc định, Tên A–Z, Thời lượng tăng dần). Dòng **Tìm thấy N bài hát**. Không có kết quả → **Không tìm thấy bài hát nào**.
5. **Tối ưu render**: kết quả lọc + sắp xếp tính bằng `useMemo` (sắp xếp trên bản sao mảng, tên dùng `localeCompare(b, "vi")`); `SongRow` bọc `React.memo`; các hàm truyền xuống `SongRow` bọc `useCallback`. Không tạo object/hàm mới trong JSX của danh sách nếu không cần.

Tự kiểm tra Phần 1

Thao tác	Kết quả mong đợi
Mở `/`	"Tìm thấy 10000 bài hát", cuộn mượt; tab Elements của DevTools chỉ có khoảng 10–15 dòng `SongRow` trong DOM
Gõ **mưa**	Trong lúc gõ danh sách chưa đổi; 300ms sau: "Tìm thấy 1250 bài hát"
Chọn thể loại **Rock** (ô tìm kiếm vẫn là "mưa")	"Tìm thấy 250 bài hát"
Xóa ô tìm kiếm, chọn Tất cả, gõ **thu hà**	"Tìm thấy 1000 bài hát" (tìm theo ca sĩ Thu Hà)
Gõ **xyz**	"Không tìm thấy bài hát nào"
Sắp xếp **Tên A–Z**, xóa bộ lọc	Dòng đầu tiên là "Biển bình yên 1"
Bật React DevTools → *Highlight updates when components render*, gõ vào ô tìm kiếm	Các dòng `SongRow` không nhấp nháy trong lúc đang gõ (chưa hết 300ms)

Phần 2 – Thanh phát nhạc và Yêu thích với Zustand (khoảng 60 phút)

Buổi 4: Zustand store với TypeScript, selector chỉ lấy đúng phần state cần dùng, middleware persist. Buổi 5: chống re-render thừa.

1. **Store phát nhạc** `src/features/player/usePlayerStore.ts` bằng `create<PlayerState>()(...)`, gồm state `queue: Song[]`, `currentIndex: number` (-1 khi chưa phát), `isPlaying: boolean`, `position: number` (giây) và các action:
   - `playQueue(songs, startIndex)`: thay hàng đợi bằng `songs`, phát từ vị trí `startIndex`, `position` về 0.
   - `togglePlay()`, `next()`, `prev()`: `next()` ở bài cuối thì dừng phát; `prev()` khi đã phát quá 3 giây thì chỉ tua về 0:00, ngược lại lùi về bài trước (bài đầu thì tua về 0:00).
   - `addToQueue(song)`: thêm vào cuối hàng đợi (bài đã có trong hàng đợi thì bỏ qua).
   - `tick()`: tăng `position` 1 giây; hết bài thì tự gọi `next()` (dùng `get()` để đọc state hiện tại).
2. **Phát từ thư viện**: bấm **Phát** ở một dòng → hàng đợi là **danh sách đang hiển thị** (đã lọc, đã sắp xếp), bắt đầu từ bài đó. Bấm **+ Hàng đợi** → thêm bài vào cuối hàng đợi.
3. **Thanh phát nhạc** (`PlayerBar`) cố định ở đáy mọi trang: **Đang phát: <tên bài> – <ca sĩ>** (chưa phát gì thì **Chưa phát bài nào**), thời gian dạng **0:05 / 2:30**, thanh tiến trình theo %, các nút **Trước**, **Phát/Tạm dừng**, **Sau**, và dòng **Hàng đợi: N bài**.

4. **Đồng hồ**: một `useEffect` dùng `setInterval` 1000ms gọi `tick()`, chỉ chạy khi `isPlaying`; cleanup bằng `clearInterval`.
5. **Chống re-render mỗi giây**: tách phần hiển thị thời gian và thanh tiến trình thành component riêng (`ProgressBar`), chỉ component này đọc `position`. Mọi component khác lấy state qua selector riêng, VD `usePlayerStore((s) ⇒ s.isPlaying)`, không lấy cả store. Khi nhạc đang chạy, **danh sách bài hát và các nút điều khiển không được render lại mỗi giây**.
6. **Yêu thích**: store riêng `src/features/favorites/useFavoritesStore.ts` (state `ids: string[]`, action `toggle(id)`), dùng middleware `persist` của Zustand với key `music-favorites` để F5 không mất. Nút ♥ trên mỗi dòng đổi giữa ♡ và ♥ (có `aria-label` **Thêm vào yêu thích** / **Bỏ yêu thích**). Trang `/favorites` liệt kê các bài đã thích (không cần ảo hóa), có nút **Phát tất cả**; chưa có bài nào → **Bạn chưa có bài hát yêu thích**.

Tự kiểm tra Phần 2

Thao tác	Kết quả mong đợi
Bấm Phát ở s1	Thanh dưới: "Đang phát: Mưa tháng Sáu 1 – Lê Minh", "0:00 / 2:30", sau 5 giây là "0:05 / 2:30"
Bấm Tạm dừng, chờ 3 giây	Thời gian đứng yên; bấm Phát thì chạy tiếp
Bấm Sau	Chuyển sang "Nắng tháng Sáu 1 – Quang Huy", thời gian về 0:00
Gõ "mưa", bấm Phát ở dòng đầu, bấm Sau	Bài tiếp theo vẫn là một bài có chữ "Mưa" (hàng đợi là danh sách đã lọc)
Bật Highlight updates khi nhạc đang chạy	Mỗi giây chỉ vùng thời gian/thanh tiến trình nhấp nháy
Bấm ♥ ở s1 và s3, mở `/favorites`, bấm F5	Vẫn còn 2 bài; ô `music-favorites` trong Local Storage có `s1`, `s3`

Phần 3 – Playlist bằng Context + useReducer, giao diện sáng/tối (khoảng 50 phút)

Buổi 4: mini-Redux bằng useReducer + Context, tách nhỏ Context, useMemo cho value, middleware. Buổi 5: code-splitting theo route.

1. **Reducer playlist** `src/features/playlists/playlistReducer.ts`: tự định nghĩa kiểu `Playlist` (`id`, `name`, `songIds`) và kiểu `PlaylistAction` là **discriminated union** gồm `create`, `rename`, `remove`, `addSong`, `removeSong`. Reducer là hàm thuần, không sửa trực tiếp state; `addSong` không thêm trùng bài.
2. **Middleware ghi log**: viết hàm `withLogger(reducer)` trả về một reducer mới, mỗi lần nhận action thì `console.log` tên action, state trước và state sau (chỉ khi `import.meta.env.DEV`), rồi trả về đúng kết quả của reducer gốc. Dùng `useReducer(withLogger(playlistReducer), ...)`.
3. **Tách 2 Context**: `PlaylistStateContext` (danh sách playlist) và `PlaylistDispatchContext` (hàm `dispatch`), cùng 2 hook `usePlaylists()` và `usePlaylistDispatch()` báo lỗi rõ ràng nếu dùng ngoài Provider. Component chỉ cần dispatch (VD form tạo playlist) dùng `usePlaylistDispatch()` để không bị render lại khi danh sách thay đổi. Danh sách playlist được lưu vào localStorage key `music-playlists` mỗi khi thay đổi.
4. **Trang** `/playlists`: form có ô **Tên playlist** và nút **Tạo**. Tên trống → **Tên playlist không được để trống**; trùng tên (không phân biệt hoa thường) → **Tên playlist đã tồn tại**. Mỗi playlist hiện tên, **N**

bài, link sang chi tiết và nút **Xóa**.

5. **Thêm bài vào playlist**: trên trang Thư viện có ô chọn **Playlist đang chọn**; mỗi dòng bài hát có thêm nút **+ Playlist** thêm bài vào playlist đang chọn (chưa có playlist nào thì nút bị khóa).
6. **Trang** `/playlists/:id`: tên playlist, các bài trong playlist kèm nút **Bỏ**, dòng **Tổng thời lượng: X phút Y giây** (dùng `reduce`), nút **Phát tất cả** (gọi `playQueue` của Zustand store). Không tìm thấy → **Không tìm thấy playlist**.
7. **Giao diện sáng/tối**: `ThemeContext` riêng (không gộp với playlist) có `theme: "light" | "dark"` và `toggleTheme`; `value` bọc `useMemo`. Nút **Giao diện tối**/**Giao diện sáng** trên thanh menu.
8. **Lazy load theo route**: 3 trang `/favorites`, `/playlists`, `/playlists/:id` dùng `React.lazy`, bọc `<Suspense fallback={<p>Đang tải trang...</p>}>`. Chạy `npm run build` phải thấy các trang này thành file JS riêng.

Tự kiểm tra Phần 3

Thao tác	Kết quả mong đợi
Tạo playlist "Chạy bộ", rồi tạo tiếp "chạy bộ"	Lần 2 báo "Tên playlist đã tồn tại"
Chọn "Chạy bộ" ở Thư viện, bấm + Playlist ở s1, s2, rồi s1 lần nữa	Playlist có 2 bài; chi tiết: "Tổng thời lượng: 5 phút 37 giây"
Mở Console khi thêm bài	Thấy log tên action `addSong` cùng state trước/sau
Bấm Phát tất cả ở chi tiết playlist	Thanh dưới phát "Mưa tháng Sáu 1", Hàng đợi: 2 bài
F5 ở trang chi tiết playlist	Playlist vẫn còn đủ bài
Mở tab Network, lần đầu bấm menu Playlist	Trình duyệt mới tải thêm file JS của trang Playlist

Phần 4 – Kiểm thử với Vitest và React Testing Library (khoảng 60 phút)

Buổi 6: unit test, test reducer và store, fake timers, renderHook, test component theo hành vi người dùng, mock function, coverage.

Viết **tối thiểu 12 test**, đặt cạnh file nguồn (`*.test.ts`, `*.test.tsx`), tên test bằng tiếng Việt mô tả hành vi. Lệnh `npm test` phải xanh toàn bộ.

File test	Các trường hợp bắt buộc
`utils/format.test.ts`	`formatDuration`: 0 → 0:00, 65 → 1:05, 300 → 5:00
`data/songs.test.ts`	`generateSongs(10000)` có 10000 phần tử, id không trùng, mọi thời lượng trong khoảng 150–300
`features/playlists/playlistReducer.test.ts`	tạo playlist; `addSong` không thêm trùng; `removeSong`; xóa playlist không tồn tại thì state giữ nguyên (`toEqual`)
`features/player/usePlayerStore.test.ts`	`playQueue` đặt đúng bài; `tick` ở giây cuối tự sang bài sau; `next` ở bài cuối thì `isPlaying` là false. Đặt lại store trước mỗi test bằng `usePlayerStore.setState(...)` trong `beforeEach`

File test	Các trường hợp bắt buộc
`hooks/useDebounce.test.ts`	`renderHook` + `vi.useFakeTimers()` : đổi giá trị, trước 300ms vẫn là giá trị cũ, sau `vi.advanceTimersByTime(300)` là giá trị mới
`features/library/SongRow.test.tsx`	hiển thị tên bài và "2:30"; bấm nút **Phát** (tìm bằng `getByRole`) thì hàm mock `vi.fn()` được gọi với đúng bài hát
`features/playlists/PlaylistsPage.test.tsx`	dùng `userEvent` : bấm Tạo khi ô trống thì thấy "Tên playlist không được để trống"; nhập tên rồi bấm Tạo thì tên xuất hiện trong danh sách (viết hàm `renderWithProviders` bọc Provider và `MemoryRouter`)

Chạy `npm run coverage` : các thư mục `src/features` và `src/utils` cần đạt **Lines ≥ 70%**. Chụp màn hình bảng coverage, lưu thành `coverage.png` ở gốc repo.

Thử thách thêm (không bắt buộc)

- Chế độ **Phát ngẫu nhiên** và **Lặp lại** trong player store, có test.
- Dùng React DevTools Profiler đo thời gian render trang Thư viện khi bỏ `FixedSizeList` (render cả 10.000 dòng) và khi dùng lại, ghi 2 con số vào `README.md` .
- Dùng middleware `devtools` của Zustand để xem lịch sử action trong Redux DevTools.
- Phím tắt: Space để Phát/Tạm dừng (nhớ `removeEventListener` khi unmount và bỏ qua khi đang gõ trong ô nhập).

Yêu cầu kỹ thuật chung

```
src/
├── App.tsx, main.tsx, types.ts, setupTests.ts
├── data/songs.ts (+ songs.test.ts)
├── utils/format.ts (+ test)
├── hooks/useDebounce.ts (+ test)
├── contexts/ThemeContext.tsx
└── features/
    ├── library/   LibraryPage.tsx, SongRow.tsx (+ test)
    ├── player/    usePlayerStore.ts (+ test), PlayerBar.tsx, ProgressBar.tsx
    ├── favorites/ useFavoritesStore.ts, FavoritesPage.tsx
    └── playlists/ playlistReducer.ts (+ test), withLogger.ts,
                   PlaylistContext.tsx, PlaylistsPage.tsx (+ test),
                   PlaylistDetailPage.tsx
```

Loại state	Công cụ bắt buộc	Lý do
Bài đang phát, hàng đợi, vị trí (đổi mỗi giây)	Zustand + selector	Đổi liên tục, cần chỉ render đúng phần đọc state
Bài hát yêu thích	Zustand + persist	State nhỏ, cần ghi nhớ khi tải lại trang
Playlist (nhiều loại action)	Context + useReducer, tách 2 Context	Logic cập nhật phức tạp, không cần thư viện ngoài

Loại state	Công cụ bắt buộc	Lý do
Giao diện sáng/tối	Context riêng + useMemo	Ít thay đổi, dùng toàn ứng dụng
Ô tìm kiếm, bộ lọc, form	useState cục bộ	Chỉ 1 component dùng

- Không dùng `any`, không dùng Redux Toolkit; mọi props và state đều có kiểu rõ ràng.
- Trang dùng `React.lazy` phải `export default`.
- `npm run build` và `npm test` đều phải chạy thành công trước khi nộp.